

CO5 ASSESSMENT 2

PSEUDOCODE WRITING TASK — ANSWERS

1. Real-Time Face Detection and Recognition

Real-Time Face Detection and Recognition

1. Problem Statement

A security system must process a live camera stream continuously, detect multiple faces, recognize registered persons using a trained dataset, and display bounding boxes with identity labels while maintaining real-time performance.

2. Objective

To design an efficient OpenCV-based face detection and recognition pipeline that can process multiple faces simultaneously with minimum processing delay.

3. Constraints

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.
- Only registered persons should be recognized.
- Unknown persons should be labeled as "Unknown".

4. Input

- Live camera/video stream.
- Trained face dataset.
- Registered person names/IDs.
- Recognition confidence threshold.

5. Detailed Pseudocode

BEGIN

 INITIALIZE camera

IF camera cannot be opened THEN

 DISPLAY "Camera Error"

 STOP

END IF

LOAD trained face recognition model

LOAD registered person names/IDs

INITIALIZE face detector

SET recognition threshold

SET frame processing parameters

WHILE camera is open DO

 CAPTURE one frame from camera

 IF frame is not captured THEN

 BREAK

 END IF

 RESIZE frame to smaller size

 STORE resized frame

 CONVERT frame from BGR to GRAYSCALE

APPLY image preprocessing

- Histogram Equalization / CLAHE
- Noise reduction if required

DETECT all faces in the frame

FOR each detected face DO

GET face bounding box

x, y, width, height

EXTRACT face region

RESIZE face to required model size

NORMALIZE face image

EXTRACT facial features

COMPARE extracted features

WITH registered face features

FIND best matching registered person

IF matching confidence is above threshold THEN

SET identity = registered person name/ID

```
ELSE
    SET identity = "Unknown"
END IF

DRAW bounding box around face

DISPLAY identity label
DISPLAY confidence if required

END FOR

DISPLAY processed frame

CHECK keyboard input

IF user presses 'q' THEN
    BREAK
END IF

END WHILE

RELEASE camera
CLOSE all OpenCV windows

END
```

6. Processing Pipeline

Camera → Frame Acquisition → Resize → Grayscale/Preprocessing → Face Detection → Face Extraction → Feature Extraction → Feature Matching → Identity Decision → Bounding Box & Label → Display

7. Output

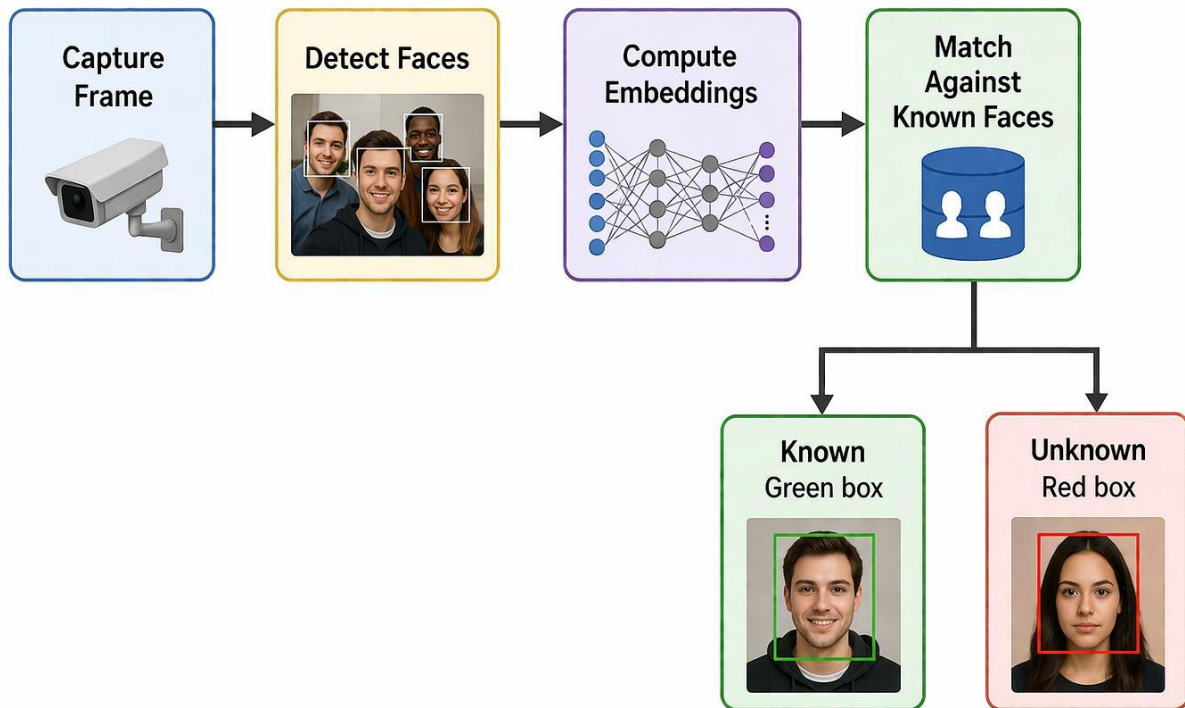
- Live video display.
- Bounding box around every detected face.
- Registered person's name/ID.
- "Unknown" label for unrecognized persons.
- Real-time recognition result.

8. Efficient Loop Control

- Resize frames before processing.
- Process only required frames when necessary.
- Use a lightweight face detector.
- Avoid unnecessary image conversions.
- Release the camera after processing.
- Use a keyboard key such as 'q' for immediate termination.

9. Justification

The system processes each camera frame through preprocessing, face detection, feature extraction, and recognition. Multiple detected faces are processed using a loop. A recognition threshold prevents incorrect identities from being assigned. Frame resizing and lightweight models reduce computational cost and help maintain near real-time performance.



2. Vehicle Detection, Tracking and Counting

Vehicle Detection, Tracking and Counting

1. Problem Statement

A traffic monitoring system must analyze a live video stream, detect vehicles entering a predefined region, track multiple vehicles across consecutive frames, count vehicles crossing a virtual line, and display bounding boxes, tracking IDs, and vehicle count.

2. Objective

To develop an efficient OpenCV-based vehicle detection, tracking, and counting system that avoids double counting and maintains near real-time performance.

3. Constraints

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.
- Track vehicles across consecutive frames.
- Count each vehicle only once.

4. Input

- Live CCTV/video stream.
- Region of Interest (ROI).
- Virtual counting line.
- Vehicle detection parameters.
- Tracking parameters.

5. Detailed Pseudocode

BEGIN

 INITIALIZE video camera

IF camera cannot be opened THEN

 DISPLAY "Video Error"

 STOP

END IF

INITIALIZE vehicle detector

INITIALIZE vehicle tracker

DEFINE Region of Interest (ROI)

DEFINE virtual counting line

SET vehicle_count = 0

CREATE empty dictionary vehicle_tracks

CREATE empty set counted_vehicle_IDs

WHILE video is running DO

 CAPTURE one frame

 IF frame is not captured THEN

 BREAK

 END IF

RESIZE frame

SELECT only ROI from frame

CONVERT ROI to GRAYSCALE if required

APPLY Gaussian/Median filtering

APPLY background subtraction

OR use a vehicle detection model

APPLY morphological operations

- Opening to remove noise
- Closing to fill gaps

DETECT vehicle contours/objects

REMOVE objects smaller than minimum area

FOR each detected vehicle DO

GET bounding box

CALCULATE vehicle centroid

ASSIGN or UPDATE tracking ID

END FOR

UPDATE vehicle tracker

FOR each tracked vehicle DO

 GET current centroid

 GET previous centroid

 CHECK whether vehicle crossed virtual line

 IF vehicle crossed the line THEN

 IF vehicle ID is NOT in counted_vehicle_IDs THEN

 INCREMENT vehicle_count

 ADD vehicle ID to counted_vehicle_IDs

 END IF

 END IF

DRAW bounding box

```
    DISPLAY vehicle tracking ID

    DISPLAY vehicle centroid

END FOR

DISPLAY vehicle_count on frame

DISPLAY virtual counting line

DISPLAY processed video

CHECK keyboard input

IF user presses 'q' THEN
    BREAK
END IF

END WHILE

RELEASE video camera
CLOSE all OpenCV windows

END
```

6. Processing Pipeline

Camera → Frame Acquisition → ROI Selection → Preprocessing → Vehicle Detection → Noise Removal → Centroid Calculation → Multi-Vehicle Tracking → Line Crossing Detection → Duplicate Check → Vehicle Count → Visualization

7. Counting Logic

IF vehicle crosses virtual line

IF vehicle_ID NOT IN counted_vehicle_IDs

vehicle_count = vehicle_count + 1

ADD vehicle_ID to counted_vehicle_IDs

END IF

END IF

This ensures that the same vehicle is **not counted more than once**.

8. Output

- Live traffic video.
- Vehicle bounding boxes.
- Unique vehicle tracking IDs.
- Virtual counting line.
- Total vehicle count.
- Real-time display.

9. Efficient Loop Control

- Resize frames to reduce processing time.
- Process only the ROI instead of the complete frame.
- Use efficient background subtraction or object detection.
- Remove small contours before tracking.
- Maintain tracking IDs instead of detecting the same vehicle repeatedly.
- Use 'q' to terminate the loop immediately.
- Release the camera and destroy OpenCV windows after termination.

10. Justification

ROI selection reduces unnecessary processing. Preprocessing and morphological operations remove noise and improve vehicle detection. Tracking maintains a unique ID for each vehicle across frames. The set of counted vehicle IDs prevents duplicate counting. Efficient frame processing and ROI-based detection help maintain near real-time performance.

